

【周末爬山】

题目描述

周末小明准备去爬山锻炼，0代表平地，山的高度使用1到9来表示，小明每次爬山或下山高度只能相差k及k以内，每次只能上下左右一个方向上移动一格，小明从左上角(0,0)位置出发

输入描述

第一行输入m n k(空格分隔)

- 代表m*n的二维山地图，k为小明每次爬山或下山高度差的最大值，

然后接下来输入山地图，一共m行n列，均以空格分隔。取值范围：

- $0 < m \leq 500$
- $0 < n \leq 500$
- $0 < k < 5$

输出描述

请问小明能爬到的最高峰多高，到该最高峰的最短步数，输出以空格分隔。

同高度的山峰输出较短步数。

如果没有可以爬的山峰，则高度和步数都返回0。

备注

所有用例输入均为正确格式，且在取值范围内，考生不需要考虑不合法的输入格式。

用例

输入	5 4 1 0 1 2 0 1 0 0 0 1 0 1 2 1 3 1 0 0 0 0 9
输出	2 2
说明	根据山地图可知，能爬到的最高峰在(0,2)位置，高度为2，最短路径为(0,0)-(0,1)-(0,2)，最短步数为2。

输入	5 4 3 0 0 0 0 0 0 0 0 0 9 0 0 0 0 0 0 0 0 0 9
输出	0 0
说明	根据山地图可知，每次爬山距离3，无法爬到山峰上，步数为0。

题目解析

本题可以使用 广度优先搜索 进行解题。

初始时，小明位于(0, 0)位置，此时走的步数step = 0，假设地图为 matrix ，则到高度matrix[0][0]的最短步数为step=0。

我们可以假设，爬山的最大高度为maxHeight，到达此高度的最小步数为minStep，则初始时，maxHeight = maxtrix[0][0]，minStep = 0。

对于任意当前位置(x, y)，假设其下一步位置是(newX, newY)



本题需要注意的是：向四个方向进行 **深搜** 时，需要注意当前位置的高度值，和下一个位置的高度值，之间的差值需要小于等于k，才能进入下一个位置继续广搜。

如果可以进入下一步，则step++，假设(newX, newY)位置的山高度为curHeight, 则：

- 如果 $curHeight > maxHeight$ 的话，则爬到了更高的山，此时直接更新 $maxHeight = curHeight$, $minStep = step$;
- 如果 $curHeight == maxHeight$ 的话，则需要继续比较step和minStep，我们需要保留step和minStep中较小步数作为 $maxHeight (==curHeight)$ 山高度的步数。
- 如果 $curHeight < maxHeight$ 的话，则无需处理

在广搜过程，为了避免走回头路，我们应该记录走过的节点，我们可以定义一个visited数组来标记走过的路。

2023.11.2

输出描述中说：如果没有可以爬的山峰，则高度和步数都返回0。

其中“没有可以爬的山峰”，结合用例2来理解的话，应该是没有可爬的更高的山峰。即当只有可爬平级的山峰，也要高度和步数都返回0。

Java算法源码

```
1 import java.util.LinkedList;
2 import java.util.Scanner;
3
4 public class Main {
5     static int m;
6     static int n;
7     static int k;
8     static int[][] matrix;
9     static int[][] offsets = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
10
11     static int maxHeight;
12     static int minStep;
```

```
13
14 | public static void main(String[] args) {
15 |     Scanner sc = new Scanner(System.in);
16
17 |     m = sc.nextInt();
18 |     n = sc.nextInt();
19 |     k = sc.nextInt();
20
21 |     matrix = new int[m][n];
22 |     for (int i = 0; i < m; i++) {
23 |         for (int j = 0; j < n; j++) {
24 |             matrix[i][j] = sc.nextInt();
25 |         }
26 |     }
27
28 |     System.out.println(getResult());
29 | }
30
31 | public static String getResult() {
32 |     // 到达matrix[0][0]高度的山峰，最短步数是0
33 |     maxHeight = matrix[0][0];
34 |     minStep = 0;
35
36 |     // 广搜
37 |     bfs();
38
39 |     if (minStep == 0) {
40 |         // 输出描述：如果没有可以爬的山峰，则高度和步数都返回0。
41 |         return "0 0";
42 |     } else {
43 |         return maxHeight + " " + minStep;
44 |     }
45 | }
46
47 | public static void bfs() {
48 |     // 广搜队列
49 |     LinkedList<int[]> queue = new LinkedList<>();
```

```

50 // 访问记录 51 | boolean[][] visited = new boolean[m][n];
52
53 // 首先是(0,0)位置进入队列，且被标记为访问过
54 queue.add(new int[] {0, 0});
55 visited[0][0] = true;
56
57 // 此时消耗步数为0
58 int step = 0;
59
60 while (queue.size() > 0) {
61     // 这里没有用queue.removeFirst来控制广搜，而是使用newQueue来控制广搜，因为这样更方便操作step
62     LinkedList<int[]> newQueue = new LinkedList<>();
63     step++;
64
65     // 遍历同一层的所有节点
66     for (int[] lastPos : queue) {
67         int x = lastPos[0];
68         int y = lastPos[1];
69
70         int lastHeight = matrix[x][y];
71
72         // 四个方向位置
73         for (int[] offset : offsets) {
74             int newX = x + offset[0];
75             int newY = y + offset[1];
76
77             // 新位置越界则无法继续广搜
78             if (newX < 0 || newX >= m || newY < 0 || newY >= n) continue;
79
80             // 新位置已访问过，则无需重复广搜
81             if (visited[newX][newY]) continue;
82
83             int curHeight = matrix[newX][newY];
84
85             // 前后位置高度差在k以内，则可以进入新位置
86             if (Math.abs(curHeight - lastHeight) <= k) {
87                 // 标记新位置已访问

```

```

88         visited[newX][newY] = true;
89     }
90     // 如果此时到达新位置高度的步数step更小，则更新对应高度的最小步数
91     if (curHeight > maxHeight || (curHeight == maxHeight && step < minStep)) {
92         maxHeight = curHeight;
93         minStep = step;
94     }
95
96     // 新位置加入下一层广搜队列
97     newQueue.add(new int[] {newX, newY});
98 }
99 }
100 }
101
102 queue = newQueue;
103 }
104 }
105 }

```

JS算法源码

```

1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      // Write your code here
7      const [m, n, k] = (await readline()).split(" ").map(Number);
8
9      const matrix = [];
10     for (let i = 0; i < m; i++) {
11         matrix.push((await readline()).split(" ").map(Number));
12     }
13
14     const offsets = [

```

运行

```

15     [-1, 0],
16     [1, 0],
17     [0, -1],
18     [0, 1],
19 ];
20
21 // 到达matrix[0][0]高度的山峰，最短步数是0
22 let maxHeight = matrix[0][0];
23 let minStep = 0;
24
25 // 广搜
26 bfs();
27
28 if (minStep == 0) {
29     // 输出描述：如果没有可以爬的山峰，则高度和步数都返回0。
30     console.log("0 0");
31 } else {
32     console.log(maxHeight + " " + minStep);
33 }
34
35 function bfs() {
36     // 广搜队列
37     let queue = [];
38     // 访问记录
39     const visited = new Array(m).fill(0).map(() => new Array(n).fill(false));
40
41     // 首先是(0,0)位置进入队列，且被标记为访问过
42     queue.push([0, 0]);
43     visited[0][0] = true;
44
45     // 此时消耗步数为0
46     let step = 0;
47
48     while (queue.length > 0) {
49         // 这里没有用queue.removeFirst来控制广搜，而是使用newQueue来控制广搜，因为这样更方便操作step
50         const newQueue = [];
51         step++;

```

```
52 |
53 | // 遍历同一层的所有节点
54 | for (let [x, y] of queue) {
55 |     const lastHeight = matrix[x][y];
56 |
57 |     // 四个方向位置
58 |     for (let [offsetX, offsetY] of offsets) {
59 |         const newX = x + offsetX;
60 |         const newY = y + offsetY;
61 |
62 |         // 新位置越界则无法继续广搜
63 |         if (newX < 0 || newX >= m || newY < 0 || newY >= n) continue;
64 |
65 |         // 新位置已访问过，则无需重复广搜
66 |         if (visited[newX][newY]) continue;
67 |
68 |         const curHeight = matrix[newX][newY];
69 |
70 |         // 前后位置高度差在k以内，则可以进入新位置
71 |         if (Math.abs(curHeight - lastHeight) <= k) {
72 |             // 标记新位置已访问
73 |             visited[newX][newY] = true;
74 |
75 |             // 如果此时到达新位置高度的步数step更小，则更新对应高度的最小步数
76 |             if (
77 |                 curHeight > maxHeight ||
78 |                 (curHeight == maxHeight && step < minStep)
79 |             ) {
80 |                 maxHeight = curHeight;
81 |                 minStep = step;
82 |             }
83 |
84 |             // 新位置加入下一层广搜队列
85 |             newQueue.push([newX, newY]);
86 |         }
87 |     }
88 | }
```



```
89 | 90 |         queue = newQueue;  
91 |     }  
92 | }  
93 | }());  
,
```

Python算法源码

```
1  # 输入获取  
2  m, n, k = map(int, input().split())  
3  matrix = [list(map(int, input().split())) for _ in range(m)]  
4  
5  offsets = ((-1, 0), (1, 0), (0, -1), (0, 1))  
6  
7  # 到达matrix[0][0]高度的山峰, 最短步数是0  
8  maxHeight = matrix[0][0]  
9  minStep = 0  
10  
11  
12  def bfs():  
13      global maxHeight, minStep  
14  
15      # 广搜队列  
16      queue = []  
17      # 访问记录  
18      visited = [[False] * n for _ in range(m)]  
19  
20      # 首先是(0,0)位置进入队列, 且被标记为访问过  
21      queue.append((0, 0))  
22      visited[0][0] = True  
23  
24      # 此时消耗步数为0  
25      step = 0  
26  
27      while len(queue) > 0:
```

```
newQueue = []30 |         step += 1

31
32 # 遍历同一层的所有节点
33 for x, y in queue:
34     lastHeight = matrix[x][y]
35
36     # 四个方向位置
37     for offsetX, offsetY in offsets:
38         newX = x + offsetX
39         newY = y + offsetY
40
41         # 新位置越界则无法继续广搜
42         if newX < 0 or newX >= m or newY < 0 or newY >= n:
43             continue
44
45         # 新位置已访问过，则无需重复广搜
46         if visited[newX][newY]:
47             continue
48
49         curHeight = matrix[newX][newY]
50
51         # 前后位置高度差在k以内，则可以进入新位置
52         if abs(curHeight - lastHeight) <= k:
53             visited[newX][newY] = True
54
55         # 如果此时到达新位置高度的步数step更小，则更新对应高度的最小步数
56         if curHeight > maxHeight or (curHeight == maxHeight and step < minStep):
57             maxHeight = curHeight
58             minStep = step
59
60         # 新位置加入下一层广搜队列
61         newQueue.append((newX, newY))
62
63     queue = newQueue
64
```

```

65 |
66 | # 算法入口
67 | def getResult():
68 |     # 广搜
69 |     bfs()
70 |
71 |     if minStep == 0:
72 |         # 输出描述: 如果没有可以爬的山峰, 则高度和步数都返回0。
73 |         return "0 0"
74 |     else:
75 |         return f"{maxHeight} {minStep}"
76 |
77 |
78 | # 算法调用
79 | print(getResult())

```

C算法源码

```

1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <stdbool.h>
4 |
5 | typedef struct ListNode {
6 |     int ele;
7 |     struct ListNode *next;
8 | } ListNode;
9 |
10 | typedef struct LinkedList {
11 |     int size;
12 |     ListNode *head;
13 |     ListNode *tail;
14 | } LinkedList;
15 |
16 | LinkedList *new_LinkedList() {
17 |     LinkedList *link = (LinkedList *) malloc(sizeof(LinkedList));

```

```
18 |     link->size = 0; 19 |     link->head = NULL;
20 |     link->tail = NULL;
21 |     return link;
22 | }
23 |
24 | void addLast_LinkedList(LinkedList *link, int ele) {
25 |     ListNode *node = (ListNode *) malloc(sizeof(ListNode));
26 |     node->ele = ele;
27 |     node->next = NULL;
28 |
29 |     if (link->size == 0) {
30 |         link->head = node;
31 |         link->tail = node;
32 |     } else {
33 |         link->tail->next = node;
34 |         link->tail = node;
35 |     }
36 |
37 |     link->size++;
38 | }
39 |
40 | int removeFirst_LinkedList(LinkedList *link) {
41 |     if (link->size == 0) exit(-1);
42 |
43 |     ListNode *removed = link->head;
44 |
45 |     if (link->size == 1) {
46 |         link->head = NULL;
47 |         link->tail = NULL;
48 |     } else {
49 |         link->head = link->head->next;
50 |     }
51 |
52 |     link->size--;
53 |
54 |     int res = removed->ele;
55 |
```

```

56     free(removed);
57
58     return res;
59 }
60
61 #define MAX_SIZE 500
62
63 int m, n, k;
64
65 int matrix[MAX_SIZE][MAX_SIZE] = {0};
66
67 bool visited[MAX_SIZE][MAX_SIZE] = {false};
68
69 int offsets[4][2] = {{-1, 0},
70                     {1, 0},
71                     {0, -1},
72                     {0, 1}};
73
74 int maxHeight;
75 int minStep;
76
77 void bfs() {
78     // 广搜队列
79     LinkedList *queue = new_LinkedList();
80
81     // 首先是(0,0)位置进入队列，且被标记为访问过
82     addLast_LinkedList(queue, 0 * n + 0);
83     visited[0][0] = true;
84
85     // 此时消耗步数为0
86     int step = 0;
87
88     // bfs
89     while (queue->size > 0) {
90         // 进入下一层需要的步数
91         step++;
92

```

```

93 // bfs按层扩散
94 |         int size = queue->size;
95 for (int i = 0; i < size; i++) {
96     int pos = removeFirst_LinkedList(queue);
97
98     int x = pos / n;
99     int y = pos % n;
100
101     int lastHeight = matrix[x][y];
102
103     // 四个方向位置
104     for (int j = 0; j < 4; j++) {
105         int newX = x + offsets[j][0];
106         int newY = y + offsets[j][1];
107
108         // 新位置越界则无法继续广搜
109         if (newX < 0 || newX >= m || newY < 0 || newY >= n) continue;
110
111         // 新位置已访问过，则无需重复广搜
112         if (visited[newX][newY]) continue;
113
114         // 前后位置高度差在k以内，则可以进入新位置
115         int curHeight = matrix[newX][newY];
116
117         if (abs(curHeight - lastHeight) <= k) {
118             // 标记新位置已访问
119             visited[newX][newY] = true;
120
121             // 如果此时到达新位置高度的步数step更小，则更新对应高度的最小步数
122             if (curHeight > maxHeight || (curHeight == maxHeight && step < minStep)) {
123                 maxHeight = curHeight;
124                 minStep = step;
125             }
126
127             // 新位置加入下一层广搜队列
128             addLast_LinkedList(queue, newX * n + newY);
129         }

```

```

130         }131     }
132     }
133 }
134
135 int main() {
136     scanf("%d %d %d", &m, &n, &k);
137
138     for (int i = 0; i < m; i++) {
139         for (int j = 0; j < n; j++) {
140             scanf("%d", &matrix[i][j]);
141         }
142     }
143
144     // 到达matrix[0][0]高度的山峰，最短步数是0
145     maxHeight = matrix[0][0];
146     minStep = 0;
147
148     // 广搜
149     bfs();
150
151     if (minStep == 0) {
152         // 输出描述：如果没有可以爬的山峰，则高度和步数都返回0。
153         puts("0 0");
154     } else {
155         printf("%d %d", maxHeight, minStep);
156     }
157
158     return 0;
159 }

```

C++ 算法源码

```

1  #include <bits/stdc++.h>
2

```

```
3 using namespace std; 4
5 int m, n, k;
6 vector<vector<int>> matrix;
7 vector<vector<bool>> visited;
8
9 int maxHeight;
10 int minStep;
11
12 int offsets[4][2] = {{0, -1},
13                     {0, 1},
14                     {-1, 0},
15                     {1, 0}};
16
17 void bfs() {
18     // 广搜队列
19     deque<int> queue;
20
21     // 首先是(0,0)位置进入队列, 且被标记为访问过
22     queue.push_back(0 * n + 0);
23     visited[0][0] = true;
24
25     // 此时消耗步数为0
26     int step = 0;
27
28     while (!queue.empty()) {
29         // 进入下一层需要的步数
30         step++;
31
32
33         // bfs按层扩散
34         int size = queue.size();
35         for (int i = 0; i < size; i++) {
36             int pos = queue.front();
37             queue.pop_front();
38
39             int x = pos / n;
40             int y = pos % n;
```



```

41
42         int lastHeight = matrix[x][y];
43
44     for (auto &offset: offsets) {
45         int newX = x + offset[0];
46         int newY = y + offset[1];
47
48         // 新位置越界则无法继续广搜
49         if (newX < 0 || newX >= m || newY < 0 || newY >= n) continue;
50
51         // 新位置已访问过，则无需重复广搜
52         if (visited[newX][newY]) continue;
53
54         // 前后位置高度差在k以内，则可以进入新位置
55         int curHeight = matrix[newX][newY];
56
57         if (abs(curHeight - lastHeight) <= k) {
58             // 标记新位置已访问
59             visited[newX][newY] = true;
60
61             // 如果此时到达新位置高度的步数step更小，则更新对应高度的最小步数
62             if (curHeight > maxHeight || (curHeight == maxHeight && step < minStep)) {
63                 maxHeight = curHeight;
64                 minStep = step;
65             }
66
67             // 新位置加入下一层广搜队列
68             queue.push_back(newX * n + newY);
69         }
70     }
71 }
72 }
73 }
74
75 int main() {
76     cin >> m >> n >> k;
77

```

```

78 | matrix = vector<vector<int>>(m, vector<int>(n));79 |     for (int i = 0; i < m; i++) {
80 |         for (int j = 0; j < n; j++) {
81 |             cin >> matrix[i][j];
82 |         }
83 |     }
84 |
85 |     visited = vector<vector<bool>>(m, vector<bool>(n, false));
86 |
87 |     maxHeight = matrix[0][0];
88 |     minStep = 0;
89 |
90 |     bfs();
91 |
92 |     if (minStep == 0) {
93 |         cout << "0 0" << endl;
94 |     } else {
95 |         cout << maxHeight << " " << minStep << endl;
96 |     }
97 |
98 |     return 0;
99 | }

```